

UNIVERSITY OF WAH(UOW)
DEPARTMENT OF COMPUTER
SCIENCE



Lab Task # 09

COURSE: Artificial Intelligence Lab_CS-202L

SUBMITTED TO: Mr. Hasssan Shafaqat

SUBMITTED BY: Muhammad Talha_CS.3ndB

Registration NO: UW-24-CS-BS-101

Task # 01

```
def dls(graph, current, goal, depth, path, visited):

    if depth == 0:
        if current == goal:
            return path + [current]
        return None
    visited.add(current)
    if current == goal:
        return path + [current]

    for neighbor in graph.get(current, []):
        if neighbor not in visited:
            result = dls(
                graph,
                neighbor,
                goal,
                depth - 1,
                path + [current],
                visited.copy()
            )
            if result:
                return result

    return None

def iddfs(graph, start, goal, max_depth=10):
    for depth in range(max_depth + 1):
        visited = set()
        path = dls(graph, start, goal, depth, [], visited)
        if path:
            return path
    return None

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}

start_node = 'A'
goal_node = 'F'
result = iddfs(graph, start_node, goal_node, max_depth=5)
print("Path found:", result)
```

Output

```
• Path found: ['A', 'C', 'F']
```

Task # 02

```
def iddfs(graph, start, goal):

    def dls(node, goal, depth, path, visited):
        if depth == 0:
            if node == goal:
                return path
            return None

        if depth > 0:
            visited.add(node)
            for neighbor in graph.get(node, []):
                if neighbor not in visited:
                    result = dls(neighbor, goal, depth - 1, path + [neighbor],
visited)
                    if result is not None:
                        return result
            visited.remove(node)
            return None

    max_depth = len(graph)

    for depth in range(max_depth):
        visited = set()
        result = dls(start, goal, depth, [start], visited)
        if result is not None:
            return result

    return None
```

Output

```
['A', 'C', 'F']
```

Task # 03

```
def one_letter_diff(word1, word2):
    diff = 0
    for a, b in zip(word1, word2):
        if a != b:
            diff += 1
    if diff > 1:
        return False
    return diff == 1
```

```

def dls_word_ladder(current, goal, word_list, depth, path, visited):
    if depth == 0:
        if current == goal:
            return path + [current]
        return None
    visited.add(current)
    if current == goal:
        return path + [current]
    for word in word_list:
        if word not in visited and one_letter_diff(current, word):
            result = dls_word_ladder(
                word,
                goal,
                word_list,
                depth - 1,
                path + [current],
                visited.copy()
            )
            if result:
                return result
    return None

def iddfs_word_ladder(start, goal, word_list, max_depth=10):
    for depth in range(max_depth + 1):
        visited = set()
        path = dls_word_ladder(start, goal, word_list, depth, [], visited)
        if path:
            return path
    return None

word_list = ['hit', 'hot', 'dot', 'dog', 'cog', 'log', 'lot']
start = 'hit'
goal = 'cog'

result = iddfs_word_ladder(start, goal, word_list)
print(result)

```

Output

```
... ['hit', 'hot', 'dot', 'dog', 'cog']
```

Task # 04

```

def dls_maze(maze, current, goal, depth, path, visited):
    rows, cols = len(maze), len(maze[0])
    r, c = current
    if depth == 0:
        if current == goal:
            return path + [current]
        return None
    visited.add(current)
    if current == goal:
        return path + [current]

```

```

directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

for dr, dc in directions:
    nr, nc = r + dr, c + dc
    next_pos = (nr, nc)

    if (0 <= nr < rows and
        0 <= nc < cols and
        maze[nr][nc] == 1 and
        next_pos not in visited):

        result = dls_maze(
            maze,
            next_pos,
            goal,
            depth - 1,
            path + [current],
            visited.copy()
        )
        if result:
            return result

return None

def iddfs_maze(maze, start, goal, max_depth=20):
    for depth in range(max_depth + 1):
        visited = set()
        path = dls_maze(maze, start, goal, depth, [], visited)
        if path:
            return path
    return None

maze = [
    [1, 0, 1, 1],
    [1, 1, 0, 1],
    [0, 1, 1, 0],
    [1, 0, 1, 1]
]

start = (0, 0)
goal = (3, 3)

result = iddfs_maze(maze, start, goal)
print(result)

```

Output

```
[(0, 0), (1, 0), (1, 1), (2, 1), (2, 2), (3, 2), (3, 3)]
```